Fatec
Baixada Santista
Rubens Lara

Aula 11 — Árvores de decisão e random forest

CART; impureza (Gini/entropia); poda; bagging; floresta aleatória

Prof. Dr. João Paulo Ferreira de Mello

26 de outubro de 2026

Ciências de Dados
Fatec Rubens Lara – Baixada Santista

O que vamos ver hoje

- **De onde viemos:** na Aula 10, o GAM somou uma curva por preditor e ganhou o placar dos 300 imóveis — porque a verdade era *aditiva*. Uma **interação** (o efeito da área depender da praia) ele só vê se você pedir.
- **O novo:** as **árvores de decisão** — perguntas de sim ou não em sequência, que separam os dados em grupos e preveem a média de cada grupo (ou a classe mais comum). Uma árvore sozinha é instável; a **média de muitas**, cada uma ajustada numa amostra bootstrap, é o **bagging** — e, sorteando preditores, a **floresta aleatória** (*random forest*).
- **Para onde vai:** a Aula 12 deixa a resposta de lado — e quando não há Y? **Agrupamento** e **componentes principais**.

O fio da aula: os **mesmos 300 imóveis** da Aula 10, agora com um prêmio de **pé na areia** — uma interação plantada de propósito.

Lembrete: o placar da Aula 10, nestes mesmos 300 imóveis

Siglas que voltam hoje. **MQO** (mínimos quadrados ordinários, Aula 4): o ajuste do `lm( )`, que minimiza a soma dos quadrados dos resíduos (**RSS**). **CV** (validação cruzada, Aula 7): $\mathrm{CV}(5)$ é o erro quadrático médio (**MSE**) de validação com 5 dobras — as *mesmas* da Aula 10. **KNN** (*k* vizinhos mais próximos, Aula 9). **GAM** (modelo aditivo generalizado, Aula 10): uma curva por preditor, somadas.

método	detalhe	$\mathrm{CV}(5)$
linha de base	prever a média	10 399
MQO	preditores retos	1 383
KNN	$k = 8$, preditores padronizados	1 357
GAM	`s(area) + s(dist) + s(idade)`	**723**
GAM com interação	`+ ti(area, dist)`	739

Em palavras: o piso é $25^2 = 625$. O GAM ganhou porque a verdade era uma *soma* de curvas: degraus na área, a queda da praia, uma reta na idade. A superfície de interação `ti( )` não ajudou — não havia interação a achar. Hoje vamos plantar uma.

Hoje: os mesmos imóveis, com um pé na areia

Mesmos 300 imóveis, mesmos preços — e um prêmio de 100 mil R$ para quem tem *as duas coisas*, área de 150 m² ou mais **e** praia a menos de 1 km: $\text{preço}_2 = \text{preço} + 100 \cdot \mathbb{1}\{\text{área} \geq 150 \text{ e dist} < 1\}$, com $\mathbb{1}\{\cdot\} = 1$ se a condição vale e 0 se não. A `cv3()` é a da Aula 10, com a base como segundo argumento:

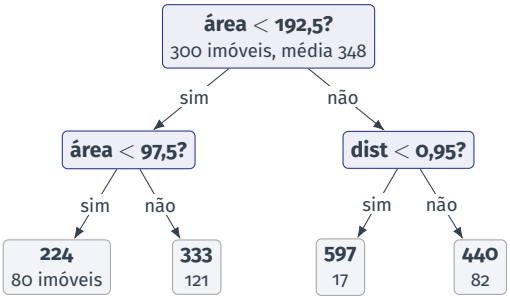
```
pe <- imov$area >= 150 & imov$dist < 1          # grande E perto da praia
imov2 <- imov; imov2$preco <- imov$preco + 100 * pe
sum(pe)                                          # quantos ganham o premio
cv3(function(tr, va) predict(gam(preco ~ s(area) + s(dist) + s(idade),
                                 data = tr), va), imov2)
cv3(function(tr, va) predict(gam(preco ~ s(area) + s(dist) + s(idade) +
                                 ti(area, dist), data = tr), va), imov2)

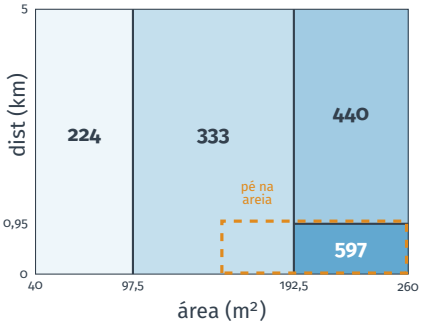

#> [1] 25
#> [1] 1153
#> [1] 897.9
```

Leia a saída: 25 imóveis ganham o prêmio. O GAM aditivo sobe de 723 para **1 153**: as curvas somadas não veem um efeito que só existe com as duas coisas juntas. Pedindo a interação (`ti()`), cai para 898 — mas, para pedir, é preciso saber o que pedir. Hoje: um modelo que acha a interação *sozinho*.

Árvores de regressão

Uma árvore é uma sequência de perguntas





Vocabulário: a primeira pergunta é a **raiz**; cada pergunta, um **nó**; as respostas, os **ramos**; as caixas do fim, as **folhas** — cada uma prevê a **média** dos preços de treino que caem nela. **Em palavras:** as quatro folhas são os quatro retângulos. À esquerda (área $< 192{,}5$) a árvore pergunta de novo pela área; à direita, pela praia: a pergunta seguinte *depende* da resposta anterior. Isso é uma interação, e ninguém precisou pedir.

O que a árvore minimiza — e como ela cresce

Definição

Árvore de regressão (CART, *classification and regression trees*; Breiman e outros, 1984): divide o espaço dos preditores em regiões retangulares $R_1, \ldots, R_J$ e prevê, em cada uma, a média $\hat{y}_{R_j}$ dos y_i de treino que caem nela. As regiões são escolhidas para minimizar

$$\mathrm{RSS} = \sum_{j=1}^{J} \sum_{i:\, x_i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2 .$$

Em palavras: testar todas as partições é impossível. A árvore escolhe *uma pergunta por vez* — a que mais derruba o RSS agora, sem olhar adiante: um algoritmo **guloso**, a **divisão binária recursiva**.

```
entrada:  dados (x(i,1), ..., x(i,p), y(i)), i = 1..n; tamanho minimo de um no
1.  comece com todos os dados num no so:  a raiz
2.  para cada preditor j e cada corte s, divida o no em {x(j) < s} e {x(j) >= s}
   e calcule o RSS dos dois lados, cada um com a sua media
3.  fique com o par (j, s) de menor RSS: sao os dois nos filhos
4.  repita 2 e 3 em cada filho, ate os nos ficarem pequenos; cada folha preve a media
```

Na mão: a primeira pergunta

O passo 2 na raiz, preditor por preditor; os cortes candidatos são os pontos médios entre valores vizinhos:

```
melhor_corte <- function(x, y) {            # testa todos os cortes de x
  xs <- sort(unique(x)); cortes <- (head(xs, -1) + tail(xs, -1)) / 2
  rss <- sapply(cortes, function(s) { e <- y[x < s]; d <- y[x >= s]
    sum((e - mean(e))^2) + sum((d - mean(d))^2) })   # os dois lados
  c(corte = cortes[which.min(rss)], RSS = min(rss) / 1e6)  # em milhoes
}
round(sapply(imov2[, c("area", "dist", "idade")], melhor_corte, y = imov2$preco), 2)
sum((imov2$preco - mean(imov2$preco))^2) / 1e6   # o RSS sem pergunta


#>         area dist idade
#> corte 192.50 0.95  15.5
#> RSS     1.91 3.45   3.6
#> [1] 3.981
```

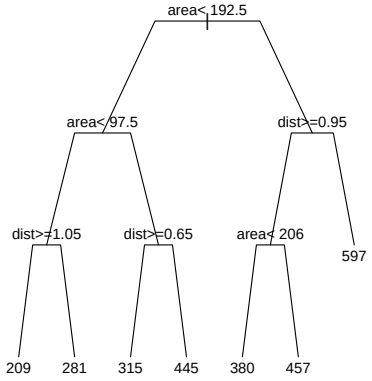
Leia a saída: sem pergunta, o RSS é 3,98 milhões (mil R$)². O melhor corte da área (192,5 m²) o derruba para 1,91 (−52%); o da praia (0,95 km), para 3,45 (−13%); o da idade, para 3,60 (−10%). A raiz é "área $<$ 192,5?" — a mesma pergunta do `rpart()`. Depois, o passo se repete dentro de cada lado.

A árvore no R: `rpart()`

O pacote `rpart` (vem com o R) faz a divisão binária recursiva. Para caber no slide, no máximo 3 andares:

```
library(rpart)
arv3 <- rpart(preco ~ area + dist + idade,
              data = imov2, maxdepth = 3)
par(mar = c(1,1,1,1), pty = "s")
plot(arv3, uniform = TRUE, branch = 0.5,
     margin = 0.03)
text(arv3, cex = 0.8, digits = 3)
```

Cada rótulo é a pergunta do nó: quem responde *sim* vai para a **esquerda**. Embaixo, a média de cada folha (mil R$).

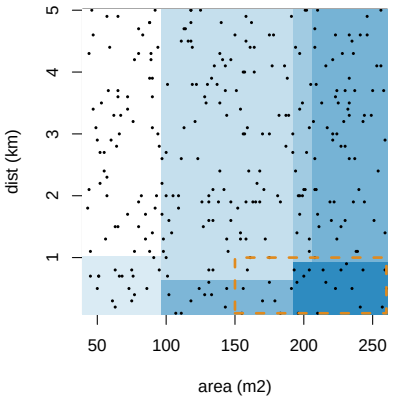


Leia a figura: a raiz pergunta pela área (192,5 m², o degrau de 200); à esquerda, de novo pela área (97,5) e só então pela praia; à direita, logo pela praia. Cada ramo tem o *seu* corte de distância (1,05, 0,65 e 0,95 km); a idade não aparece.

As folhas no plano: a árvore achou o pé na areia

Cada folha é um retângulo no plano área $\times$ distância (a idade não entra nesta árvore). Pintamos o plano pela previsão:

```
ga <- seq(40, 260, length = 150)
gd <- seq(0.1, 5, length = 150)
z  <- outer(ga, gd, function(a, d) predict(arv3,
        data.frame(area = a, dist = d, idade = 25)))
par(mar = c(4,4,1,1), pty = "s")
image(ga, gd, z, xlab = "area (m2)", ylab = "dist (km)",
      col = colorRampPalette(c("white", cazul))(30))
points(imov2$area, imov2$dist, pch = 16, cex = 0.4)
rect(150, 0.1, 260, 1, border = claranja,   # o pe
     lwd = 2.5, lty = 2)                     # na areia
```



Leia a figura: os cortes da área atravessam o plano; os da praia, não. Perto da praia, o salto é de 72 mil R$ nos pequenos (281 − 209), 130 nos médios (445 − 315) e 157 nos grandes (597 − 440): a árvore achou o pé na areia (o tracejado) sozinha — em retângulos aproximados.

Até onde crescer? A árvore cheia decora o treino

Sem freio, a árvore cresce até não ter o que separar. O `rpart()` valida *todas as podas* de uma vez (`xval = dobra3`: as nossas dobras); a `cptable` dá o erro de treino e o de validação de cada tamanho:

```
cheia <- rpart(preco ~ area + dist + idade, data = imov2, cp = 0,
               minsplit = 2, minbucket = 1, xval = dobra3)   # sem freio
c(folhas = sum(cheia$frame$var == "<leaf>"), treino = mse(predict(cheia), imov2$preco))
tab <- cheia$cptable; s2 <- mean((imov2$preco - mean(imov2$preco))^2)
folhas <- unname(tab[, "nsplit"]) + 1          # uma linha por subarvore
treino <- unname(tab[, "rel error"]) * s2; cvmse <- unname(tab[, "xerror"]) * s2
round(c(cv_cheia = tail(cvmse, 1), cv_melhor = min(cvmse),
        folhas_melhor = folhas[which.min(cvmse)]))


#> folhas treino
#>    294      0
#>      cv_cheia     cv_melhor folhas_melhor
#>          2008          1677            36
```

Leia a saída: 294 folhas e erro de treino **zero**: a árvore decorou os 300 preços (seis folhas juntam dois imóveis de mesmo preço). Validada, a cheia erra 2 008; a melhor poda, com 36 folhas, 1 677 — a curva em U da Aula 6.

A poda: um pedágio por folha

Definição

Poda por custo-complexidade: para cada $\alpha \geq 0$, a subárvore T da árvore cheia que minimiza

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i:\, x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha\,|T|, \qquad |T| = \text{número de folhas.}$$

Em palavras: o pedágio de novo (Aulas 8 e 10) — cada folha paga α. Com $\alpha = 0$ fica a árvore cheia; com α enorme, só a raiz. Uma folha nova só fica se baixar o RSS em mais do que α.

```
1.  cresca uma arvore grande, sem freio
2.  para alfa crescente, corte primeiro os galhos que mais custam folhas e menos
    baixam o RSS: uma sequencia de subarvores, da cheia ate a raiz
3.  escolha alfa por validacao cruzada (em cada dobra, a arvore cresce e e
    podada so com o treino da dobra)
4.  devolva a subarvore do alfa escolhido, com todos os dados
```

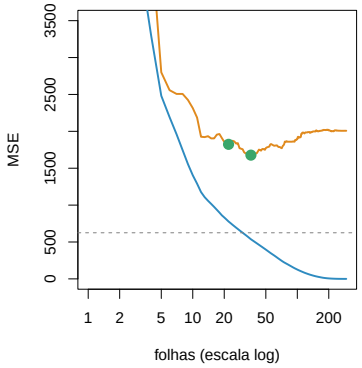
No R, o botão é o cp, em fração do RSS da raiz ($\alpha = \text{cp} \times \text{RSS(raiz)}$); `prune(arvore, cp = ...)` poda.

A poda no R: o fundo do U e a regra de um erro padrão

```
i_min <- which.min(cvmse)                       # o fundo da CV
lim   <- cvmse[i_min] + tab[i_min, "xstd"] * s2  # + 1 erro padrao
i_1ep <- min(which(cvmse <= lim))               # a menor abaixo do limite
podada <- prune(cheia, cp = tab[i_1ep, "CP"])
rbind(folhas = folhas[c(i_min, i_1ep)], cv = round(cvmse[c(i_min, i_1ep)]))

#>        [,1] [,2]
#> folhas   36   22
#> cv     1677 1824
```

```
par(mar = c(4,4,1,1), pty = "s")
plot(folhas, cvmse, log = "x", type = "l", lwd = 2, col = claranja,
     ylim = c(0, 3500), xlab = "folhas (escala log)", ylab = "MSE")
lines(folhas, treino, lwd = 2, col = cazul)              # treino
abline(h = 625, lty = 2, col = "gray50")                 # o piso
points(folhas[c(i_min, i_1ep)], cvmse[c(i_min, i_1ep)],
       pch = 16, cex = 1.6, col = cverde)
```



Leia: o treino (azul) cai até zero; a validação (laranja) desce até 36 folhas e volta a subir. A regra de um erro padrão (Aula 8) fica com **22 folhas** (1 824): quase o mesmo erro, árvore menor. Mesmo podada, a árvore erra mais que o GAM aditivo (1 153): o preço é uma soma de curvas suaves, e degraus precisam de muitas folhas para imitá-las.

Árvores de classificação

Resposta qualitativa: o imóvel é caro?

Nova resposta: **caro** = preço acima de 400 mil R$ (92 dos 300 imóveis, 30,7%). Cada folha prevê a **classe mais comum** entre os seus imóveis; $\hat{p}_{mk}$ é a proporção da classe k na folha m. Para escolher a pergunta, o RSS dá lugar a uma **impureza**:

Definição

Numa folha com proporções $\hat{p}_{m1}, \ldots, \hat{p}_{mK}$:

$$E = 1 - \max_k \hat{p}_{mk} \text{ (erro de classificação)}, \qquad G = \sum_k \hat{p}_{mk}\,(1 - \hat{p}_{mk}) \text{ (índice de Gini)},$$

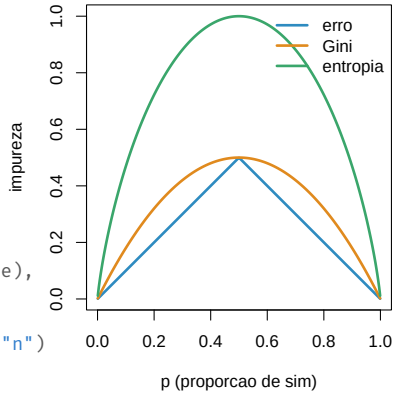
$$D = -\sum_k \hat{p}_{mk}\,\log_2 \hat{p}_{mk} \text{ (entropia, em bits)}.$$

Em palavras: as três valem **zero** numa folha pura (todo mundo da mesma classe) e são máximas com as classes meio a meio. Uma divisão vale a **média das impurezas dos filhos, ponderada pelo tamanho**; a queda em relação ao pai é o **ganho** — com a entropia, o *ganho de informação*. A árvore escolhe a pergunta de maior ganho.

As três impurezas com duas classes

Com duas classes e p a proporção de "sim" na folha:
$E = \min(p, 1-p)$, $G = 2p(1-p)$ e
$D = -p\log_2 p - (1-p)\log_2(1-p)$.

```
p <- seq(0.001, 0.999, length = 300)
erro    <- pmin(p, 1 - p)
gini    <- 2 * p * (1 - p)
entrop  <- -p * log2(p) - (1 - p) * log2(1 - p)
par(mar = c(4,4,1,1), pty = "s")
matplot(p, cbind(erro, gini, entrop), type = "l",
        lty = 1, lwd = 2.5, col = c(cazul, claranja, cverde),
        xlab = "p (proporcao de sim)", ylab = "impureza")
legend("topright", c("erro", "Gini", "entropia"),
       lwd = 2.5, col = c(cazul, claranja, cverde), bty = "n")
```



Leia a figura: as três sobem de zero ($p = 0$, folha pura de "não") até o máximo em $p = 0{,}5$ e voltam a zero ($p = 1$). O erro é um bico, feito de retas; Gini e entropia são *curvas*, que continuam caindo enquanto a folha fica mais pura — e é isso que desempata o próximo slide.

Por que não o erro? Um empate que o Gini desfaz

Exemplo

Um nó com 400 “sim” e 400 “não”. Duas divisões candidatas: **A** = (300 sim, 100 não) e (100 sim, 300 não); **B** = (200 sim, 400 não) e (200 sim, 0 não).

	erro	Gini	entropia (bits)
o nó pai	0,500	0,500	1,000
divisão A	0,250	0,375	0,811
divisão B	0,250	**0,333**	**0,689**

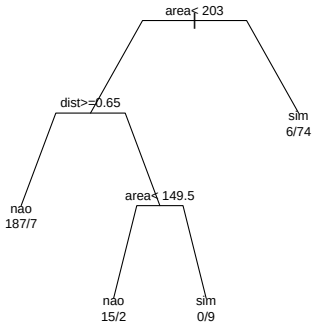
Na B, o filho de 600 tem $p = \frac{1}{3}$: $G = 2 \cdot \frac{1}{3} \cdot \frac{2}{3} = 0{,}444$ e $D = 0{,}918$; o de 200 é puro ($G = D = 0$). Ponderando por $\frac{600}{800}$: $G = 0{,}333$ e $D = 0{,}689$. Na A, os dois filhos têm $p = \frac{3}{4}$: $G = 2 \cdot \frac{3}{4} \cdot \frac{1}{4} = 0{,}375$.

Em palavras: o erro só olha a classe vencedora e **empata** (200 erros em 800 nas duas). Gini e entropia preferem a B, que criou uma **folha pura**: 200 imóveis que dispensam mais perguntas. A árvore *cresce* pelo Gini (padrão do `rpart()`) ou pela entropia (`parms = list(split = "information")`); o erro mede a árvore pronta.

A árvore de classificação: “o imóvel é caro?”

```
imov2$caro <- factor(ifelse(imov2$preco > 400,
                            "sim", "nao"))
arvc <- rpart(caro ~ area + dist + idade,
              data = imov2, method = "class")
par(mar = c(1,1,1,1), pty = "s")
plot(arvc, uniform = TRUE, branch = 0.5,
     margin = 0.12)
text(arvc, use.n = TRUE, cex = 0.75, digits = 3)
# use.n: quantos "nao"/"sim" na folha
```

O `rpart()` já vem com freio: cp $= 0{,}01$.



Leia a figura: com 203 m² ou mais, 74 de 80 são caros. Abaixo disso, quase ninguém — a não ser **perto da praia** (dist $< 0{,}65$) **e** com **149,5 m² ou mais**: **9 de 9** caros. É o pé na areia. Na validação cruzada (mesmas dobras), a árvore erra 8,7%; chutar “não caro”, 30,7%; a logística da Aula 5, 6,7% — erra menos, mas não conta a história.

Muitas árvores: bagging e floresta aleatória

Uma árvore só é instável

Seis amostras **bootstrap** (Aula 7: sortear 300 imóveis *com reposição*) e, em cada uma, a árvore padrão do `rpart()`. Todas preveem o mesmo imóvel `x0`: 180 m², a 500 m da praia, 20 anos (verdade: 509,4 mil R$).

```
x0 <- data.frame(area = 180, dist = 0.5, idade = 20)
res <- sapply(1:6, function(s) {
  set.seed(s); i <- sample(300, replace = TRUE)      # uma amostra bootstrap
  a <- rpart(preco ~ area + dist + idade, data = imov2[i, ])
  c(corte_raiz = a$splits[1, "index"], previsao = unname(predict(a, x0))) })
round(res, 1)                                        # uma coluna por amostra

#>             [,1]  [,2]  [,3]  [,4]  [,5]  [,6]
#> corte_raiz 195.0 199.5 178.5 169.0 169.0 192.5
#> previsao   443.5 518.3 592.8 504.7 511.6 451.6
```

Leia a saída: o corte da raiz anda de 169 a 199,5 m², e o imóvel de 180 m² cai ora de um lado, ora do outro. A previsão vai de 443,5 a 592,8 mil R$. Em 200 amostras, o desvio padrão da previsão é 60 mil R$: a árvore tem **variância alta**. Mudou um pouco a amostra, mudou a árvore.

Bagging: a média de muitas árvores

Para B previsões, cada uma com variância σ^2 e correlação ρ entre pares:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x_0)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Em palavras: com B grande, o segundo termo some; o primeiro fica — árvores parecidas erram *juntas*. Se fossem independentes ($\rho = 0$), a variância cairia para σ^2/B.

```
bagging (bootstrap aggregating; Breiman, 1996) — entrada:  dados de treino; B
1.  para b = 1..B: sorteie n observacoes com reposicao (uma amostra bootstrap)
   e cresca nela uma arvore grande, sem podar
2.  para prever em x0:  a media das B previsoes (regressao)
   ou a classe mais votada (classificacao)
```

Em palavras: cada árvore grande tem pouco viés e muita variância; a média mata a variância. Não há poda: o freio agora é a média. No imóvel x0: uma árvore oscila com desvio padrão de 60 mil R$; a média de 500 árvores (o bagging que ajustaremos daqui a pouco) prevê 525,2 — a verdade é 509,4.

Floresta aleatória: sortear preditores para descorrelacionar

No bagging, a área — o preditor mais forte — está na raiz de *todas* as árvores: elas ficam parecidas, e o ρ não cai. A **floresta aleatória** (*random forest*; Breiman, 2001) muda uma linha:

```
igual ao bagging, mas em CADA no:  sorteie m dos p preditores e procure a melhor
pergunta so entre eles (padrao:  m = p/3 na regressao, raiz de p na classificacao;
com m = p, volta a ser o bagging)
```

500 árvores nos 300 imóveis	árvores com a área na raiz	correlação média entre as árvores
$m = 3$ (bagging)	100%	0,93
$m = 2$	63%	0,88
$m = 1$	38%	0,74

Em palavras: sem a área na mão, a árvore é obrigada a começar pela praia ou pela idade; as árvores ficam diferentes entre si, e a média ganha mais. O preço: cada árvore fica *pior* (com $m = 1$, um terço das perguntas é sobre a idade, que pouco explica). O m é um **botão** — e quem o escolhe é o erro fora do saco, a seguir.

O erro fora do saco (OOB): validação de graça

Numa amostra bootstrap de tamanho n, a chance de uma observação *não* ser sorteada nenhuma vez é

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow[n\to\infty]{} e^{-1} \approx 0{,}368 \qquad (\text{com } n = 300\text{: } 0{,}3673).$$

Em palavras: cada árvore deixa de fora cerca de um terço dos imóveis. Para cada imóvel, junte só as árvores que *não* o viram e preveja com elas: é a previsão **fora do saco** (*out-of-bag*, **OOB**). O erro OOB é uma validação que vem de graça, sem dobras e sem separar dados.

Exemplo

Com $n = 300$ e $B = 500$ árvores, cada imóvel fica de fora de $500 \times 0{,}367 \approx 184$ árvores, em média. Na floresta de $m = 2$ do próximo slide: 184,0, de 155 a 220.

É a conta do 0,632 que a Aula 7 pulou: cada amostra bootstrap tem, em média, $1 - 0{,}368 = 63{,}2\%$ de observações distintas — o resto são repetições.

A floresta no R: `randomForest()`

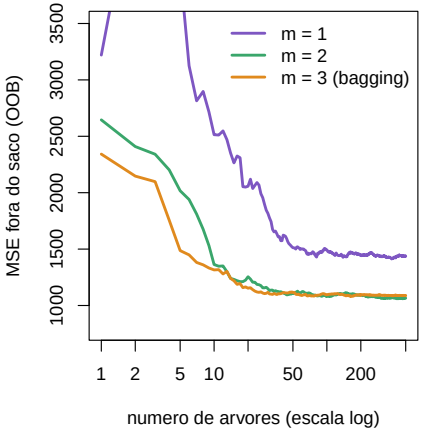
O pacote `randomForest` (do CRAN) ajusta bagging e floresta: o botão `mtry` é o m, `ntree` é o B. Três florestas, $m = 1, 2$ e 3, e o erro OOB à medida que as árvores entram:

```
library(randomForest)
florestas <- lapply(1:3, function(m) { set.seed(5)
  randomForest(preco ~ area + dist + idade, data = imov2, mtry = m,
               ntree = 500, importance = TRUE) })
round(sapply(florestas, function(f) tail(f$mse, 1)))   # OOB com 500


#> [1] 1437 1065 1087
```

```
par(mar = c(4,4,1,1), pty = "s")
matplot(sapply(florestas, function(f) f$mse), type = "l", lty = 1,
        lwd = 2.5, col = c(croxo, cverde, claranja), log = "x",
        ylim = c(800, 3500), xlab = "numero de arvores (escala log)",
        ylab = "MSE fora do saco (OOB)")
legend("topright", c("m = 1", "m = 2", "m = 3 (bagging)"), lwd = 2.5,
       col = c(croxo, cverde, claranja), bty = "n")
```

Mais árvores não sobreajustam; o botão é o m



Leia a figura: com uma árvore, o erro OOB é o de uma árvore só (acima de 2 300 nas três). Com 50 árvores, já caiu à metade; de 100 em diante, as curvas ficam planas. **Mais árvores não sobreajustam** — só estabilizam a média; o B é custo de computação, não botão.

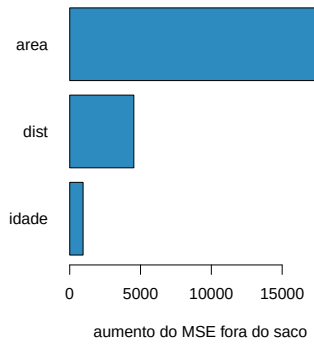
O botão é o m: o erro OOB com 500 árvores é 1 437 ($m = 1$), **1 065** ($m = 2$) e 1 087 ($m = 3$, o bagging). Fica o $m = 2$. O padrão do `randomForest()` ($p/3 = 1$) é aqui o pior: com só 3 preditores, sortear tem pouco a descorrelacionar e muito a perder. Com dezenas de preditores (como no seu banco), o sorteio pesa mais.

Abrindo a floresta: a importância dos preditores

Embaralhe uma coluna nos imóveis fora do saco — isso quebra a ligação dela com o preço — e meça quanto o erro OOB sobe:

```
rf <- florestas[[2]]                  # a floresta de m = 2
# quanto o MSE fora do saco sobe ao embaralhar:
imp <- importance(rf, scale = FALSE)[, 1]
round(imp)
par(mar = c(4,5,1,1), pty = "s")
barplot(sort(imp), horiz = TRUE, las = 1, col = cazul,
        xlab = "aumento do MSE fora do saco")

#>  area  dist idade
#> 18652  4527   943
```



Leia: embaralhar a área faz o erro OOB subir 18 652; a distância, 4 527; a idade, 943. A importância diz *quanto* a floresta usa cada preditor — não diz *para que lado* ele empurra o preço, nem que ele *causa* alguma coisa. `varImpPlot(rf)` desenha esta e a outra medida do pacote (a queda total de RSS nas perguntas sobre o preditor).

O placar em duas verdades

$\mathrm{CV}(5)$ com as mesmas dobras; árvore, KNN e floresta no melhor botão:

método	botão	verdade aditiva (Aula 10)	com o pé na areia
linha de base	prever a média	10 399	13 346
MQO	—	1 383	2 264
KNN	$k = 8$ e $k = 6$	1 357	1 729
GAM	—	**723**	1 153
GAM + `ti(area, dist)`	—	739	**898**
árvore podada	21 e 36 folhas	1 730	1 677
bagging	$m = 3$	1 027	1 026
floresta aleatória	$m = 2$	993	1 089

Leia: a árvore sozinha perde até para o MQO na verdade aditiva; a média de muitas corta o erro dela quase à metade. Com o pé na areia, bagging e floresta passam o GAM aditivo sem saber onde está a interação — mas o GAM a quem se *pede* a interação certa ganha de todos. Sem interação, ganha o GAM. **Não há campeão:** o melhor modelo depende da verdade, que fora da simulação ninguém conhece — quem decide é a validação cruzada, **no seu banco**.

Exercícios e trabalho

Exercícios — cálculo

Exercício

Áreas 50, 70, 90, 120, 150 e 200 m²; preços 200, 220, 260, 300, 310 e 420 mil R$. Calcule o RSS dos cortes em 80, 135 e 175 m². Qual a árvore escolhe?

Exercício

Um nó tem 30 caros e 10 não: calcule erro, Gini e entropia. Compare as divisões (20, 0) + (10, 10) e (25, 5) + (5, 5) pelo erro e pelo Gini ponderados.

Exercício

Calcule $(1 - 1/n)^n$ para $n = 10$ e $n = 1\,000$. Com 300 árvores e $n = 1\,000$, de quantas árvores cada observação fica de fora, em média?

Exercício

Subárvores de 1, 4 e 12 folhas têm RSS 900, 300 e 200. Qual minimiza $\mathrm{RSS} + \alpha|T|$ com $\alpha = 10$, 50 e 300? Desde que α a de 4 vence a de 12?

Exercícios — conceituais

Exercício (Guloso)

Por que a árvore não testa todas as partições? Invente um preço em que a melhor *primeira* pergunta não leva à melhor árvore (dica: um prêmio só com as duas condições juntas).

Exercício (Interação de graça)

Por que a árvore pega interações sem que você peça, e o GAM não?

Exercício (Mais árvores $\times$ árvore maior)

Por que mais árvores no bagging não sobreajustam, e mais folhas numa árvore sim?

Exercício (Sortear)

Por que a floresta sorteia preditores em cada nó? Por que, com $p = 3$, ajudou pouco?

Exercício (Importância não é causa)

A área é o preditor mais importante da floresta. Então ampliar um imóvel aumenta o preço dele? O que a importância *não* diz?

Laboratório — no seu banco de dados

Resposta *y* e os preditores da **ficha da sua pergunta** (Entrega 1), quantos quiser: árvore e floresta aceitam preditores qualitativos (`factor`) sem criar colunas. Mesmas dobras do laboratório da Aula 10:

```
library(rpart); library(randomForest)
d <- na.omit(data.frame(y = dados$y, x1 = dados$x1, x2 = dados$x2, x3 = dados$x3))
# banco grande (milhares de linhas)? trabalhe numa amostra: d <- d[sample(nrow(d), 5000), ]
set.seed(1); dobra <- sample(rep(1:5, length = nrow(d)))
cheia <- rpart(y ~ ., data = d, cp = 0.001, xval = dobra)             # cresce e valida
tab <- cheia$cptable; i <- which.min(tab[, "xerror"])
tab[i, "xerror"] * mean((d$y - mean(d$y))^2)                          # CV(5) da melhor poda
arv <- prune(cheia, cp = tab[i, "CP"])
par(pty = "s"); plot(arv, uniform = TRUE, margin = 0.1); text(arv, cex = 0.7)
p <- ncol(d) - 1; ms <- unique(pmax(1, round(c(sqrt(p), p/3, p/2, p))))   # m candidatos
oob <- sapply(ms, function(m) { set.seed(1); tail(randomForest(y ~ ., data = d, mtry = m)$mse, 1) })
set.seed(1); rf <- randomForest(y ~ ., data = d, mtry = ms[which.min(oob)], importance = TRUE)
rf; varImpPlot(rf, scale = FALSE)                                      # o erro OOB e a importancia
```

Entregar (código em `estrutura/codigo/`, registro em `consolidados/`): o erro da árvore podada e o OOB da floresta, ao lado da linha de base, do MQO e do GAM da Aula 10; a figura da árvore podada e a da importância; *uma frase* com a primeira pergunta da árvore e se alguma pergunta depende de outra (interação?); e **o que isso muda para a sua pergunta**. Árvore grande demais? `maxdepth = 3` só para o desenho. Resposta binária: veja a lista.

Resumo

- **Árvore:** perguntas de sim ou não em sequência; cada folha prevê a média (ou a classe mais comum). Cresce gulosa, pelo RSS ou pelo Gini/entropia — e pega **interações** sem que se peça.
- **Poda:** um pedágio α por folha, escolhido por validação cruzada; sem ela, a árvore decora o treino.
- **Bagging e floresta:** a média de muitas árvores em amostras bootstrap mata a variância; a floresta sorteia *m* preditores por nó para descorrelacioná-las. O erro **OOB** valida de graça e escolhe o *m*.
- **Uma árvore se lê; uma floresta prevê melhor** — e se explica pela importância dos preditores. Quem decide é a validação cruzada.

Próxima aula: e quando não há resposta *Y*? **Agrupamento** (*K*-médias e hierárquico) e **componentes principais** (PCA, *principal component analysis*): achar estrutura nos dados sem ninguém dizer o que prever.